Lecture 9: Grover's algorithm

Problem: Let $f:\{0,1\}^n \to \{0,1\}$ s.t.

$$f(x) = \begin{cases} 1 & \text{if } x=a \text{ for some } a \in \{0,1\}^n \\ 0 & \text{otherwise} \end{cases}$$

Searching through 2^n different entries classically:

- sorted data: $\log_2(2^n) = n$ queries using binary search
- unsorted data: 2^n queries by brute force

Quantum brute-force search: $\sqrt{2^n}$ queries.

More generally, N versus $\sqrt{N}$ queries when searching through N items.

Oracle: $O_f|x\rangle = (-1)^{f(x)}|x\rangle = \begin{cases} -|x\rangle & \text{if } x=a \\ |x\rangle & \text{otherwise} \end{cases}$

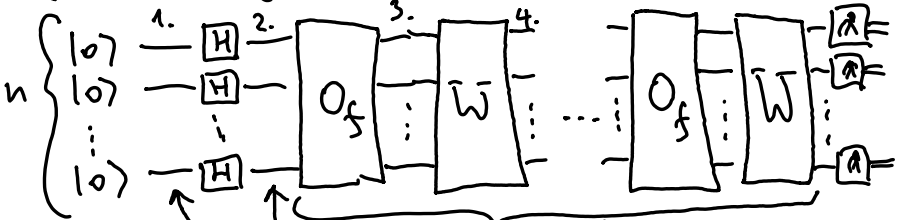
Example 1 Explicitly storing the password for a user: $f(x) = \begin{cases} 1 & \text{if } x = \text{"password"} \\ 0 & \text{otherwise} \end{cases}$

is problematic in case the source code of f gets stolen.

Example 2 Storing the hash (e.g. SHA) of a password: $f(x) = \begin{cases} 1 \text{ if } h(x) = \text{"73A0F2BE5"} \\ 0 \text{ otherwise} \end{cases}$

The hash function h is easy to compute but hard to invert. A hacker cannot easily recover the password even if the data is stolen. The source code of f contains only the source code of the hash function h, which usually is already public, and the string "73A0F2BE5" which does not reveal the password in a straightforward way. The only way to crack it is by brute force.

Grover's algorithm Let $N = 2^n$ be the size of the set we are searching in.



1. $|0\rangle^{\otimes n}$
2. $|+\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle$
3. $\frac{1}{\sqrt{2^n}} \left(\left(\sum_{x \in \{0,1\}^n} |x\rangle \right) - 2|a\rangle \right)$

We want to get $a \in \{0,1\}^n$ as the answer, so we want to amplify the amplitude of $|a\rangle$

$$O_f|x\rangle = \begin{cases} -|x\rangle & \text{if } x = a \\ |x\rangle & \text{otherwise} \end{cases}$$

Can check that:

$$O_f = I - 2|a\rangle\langle a|$$

$$O_f = \begin{matrix} \\ a \end{matrix} \begin{pmatrix} 1 & & & \\ & \ddots & & \\ & & -1 & \\ & & & \ddots \\ & & & & 1 \end{pmatrix} \begin{matrix} a \\ \\ \\ \\ \\ \end{matrix}$$

Main idea: two consecutive reflections make a rotation.

Let $|\Psi\rangle = |+\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle$ be the uniform superposition and let $W = -(I - 2|\Psi\rangle\langle\Psi|)$ be the reflection with respect to $|\Psi\rangle$.
Let $G = W O_f$ be one Grover iteration.
The final state after N iterations is

$$G^{\sqrt{N}} H^{\otimes n} |0\rangle^{\otimes n} = G^{\sqrt{N}} |+\rangle^{\otimes n} = G^{\sqrt{N}} |\Psi\rangle.$$

Let's analyze how one step of the Grover iteration $G = W O_f$ affects the state. Recall that

$$W = 2|\Psi\rangle\langle\Psi| - I \qquad O_f = I - 2|a\rangle\langle a|$$

Note that $\langle\Psi|a\rangle = \langle a|\Psi\rangle = 1/\sqrt{2^n} = 1/\sqrt{N}$. Consider how W and O_f act on $|\Psi\rangle$ and $|a\rangle$:

$$W|\Psi\rangle = |\Psi\rangle \qquad O_f|\Psi\rangle = |\Psi\rangle - \frac{2}{\sqrt{N}}|a\rangle$$

$$W|a\rangle = \frac{2}{\sqrt{N}}|\Psi\rangle - |a\rangle \qquad O_f|a\rangle = -|a\rangle$$

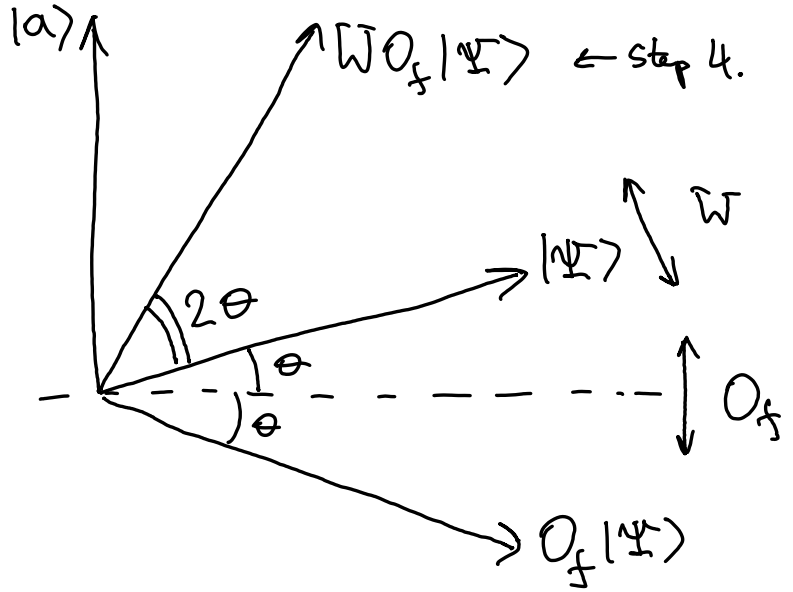
So starting from the state $|\Psi\rangle$ and applying any sequence of O_f and W will still give you a real linear combination of $|a\rangle$ and $|\Psi\rangle$, i.e., the state remains in the 2-dim subspace:

$$\alpha|\Psi\rangle + \beta|a\rangle \quad \text{for some } \alpha, \beta \in \mathbb{R}$$

Geometric view of Grover's algorithm

- Since $O_f|a\rangle = -|a\rangle$, O_f reflects about the line $\perp$ to $|a\rangle$.
- Since $W|\Psi\rangle = |\Psi\rangle$, W is a reflection about $|\Psi\rangle$.

Recall that $\langle a|\Psi\rangle = 1/\sqrt{N}$, so $|a\rangle$ and $|\Psi\rangle$ are almost orthogonal.



Exercise:

The composition of two reflections is a rotation.

If the angle between the axes is θ, the angle of rotation is 2θ.

Analysis of complexity

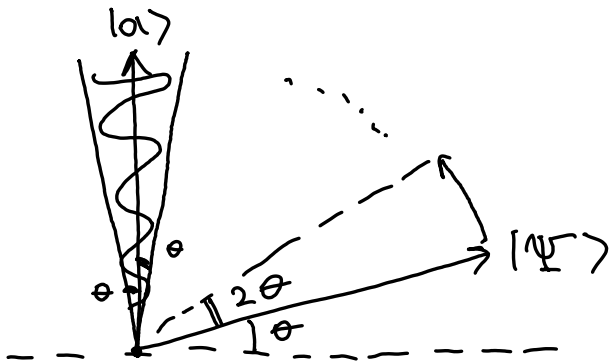
The Grover iteration $G = W O_f$ is a rotation by angle 2θ from $|\Psi\rangle$ to $|\alpha\rangle$, where the angle between $|\Psi\rangle$ and $|\alpha\rangle$ is $\pi/2 - \theta$. Note that

$$\sin\theta = \cos\left(\frac{\pi}{2} - \theta\right) = \langle\alpha|\Psi\rangle = 1/\sqrt{N}.$$

If N is large, $|\Psi\rangle$ and $|\alpha\rangle$ are nearly orthogonal so θ is small and

$$\theta \approx \sin\theta = 1/\sqrt{N}.$$

After $t = \frac{\pi/2}{2\theta} \approx \frac{\pi}{4}\sqrt{N}$ iterations of G, the final state $G^t|\Psi\rangle$ is within angle θ of $|\alpha\rangle$:



Measuring in the comp. basis yields $a \in \{0,1\}^n$ w.p.

$$|\langle a|G^t|\Psi\rangle|^2 \geq (\cos\theta)^2 = 1 - (\sin\theta)^2 = \frac{N-1}{N} \approx 1$$

when N is large. Note that further iterations beyond t decrease the probability of success.

Multiple solutions:

Grover's alg works even if the solution $a \in \{0,1\}^n$ is not unique. Let $A \subseteq \{0,1\}^n$ be a set of possible solutions, $M = |A|$.

Then Grover's iteration is a rotation in the 2-dim subspace spanned by

$$|\Sigma\rangle = \frac{1}{\sqrt{N}} \sum_{x \in \{0,1\}^n} |x\rangle \quad \text{and} \quad |A\rangle = \frac{1}{\sqrt{M}} \sum_{a \in A} |a\rangle.$$

The angle θ between those vectors is smaller, so the required number of iterations (but also the success prob.) is also smaller.

The number of iterations: $t = O(\sqrt{N/M})$.

Prob. of success: $1 - \frac{M}{N}$.

Implementing $\tilde{W}$

$\tilde{W} = 2|\Psi\rangle\langle\Psi| - I, \quad |\Psi\rangle = |+\rangle^{\otimes n}.$

Note that $H^{\otimes n}|\Psi\rangle = |0\rangle^{\otimes n}$

$$H^{\otimes n} \tilde{W} H^{\otimes n} = 2|0^n\rangle\langle 0^n| - I$$

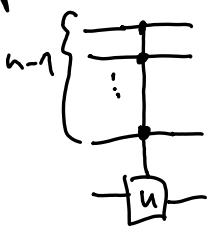
$$= \begin{pmatrix} 1 & & & \\ & -1 & & \\ & & -1 & \\ & & & \ddots \\ & & & & -1 \end{pmatrix} \leftarrow 00\ldots0$$

Conjugate further by $X^{\otimes n}$:

$$-X^{\otimes n} H^{\otimes n} \tilde{W} H^{\otimes n} X^{\otimes n} = -\left(2|1^n\rangle\langle 1^n| - I\right) = I - 2|1^n\rangle\langle 1^n|$$

$$= \begin{pmatrix} 1 & & & \\ & 1 & & \\ & & \ddots & \\ & & & 1 \\ & & & & -1 \end{pmatrix} = C_{n-1}(Z) \leftarrow 11\ldots1$$

where $C_{n-1}(U)$ is the multiply controlled U operation:



$$C_{n-1}(U) = \begin{pmatrix} 1 & & & \\ & 1 & & 0 \\ & & \ddots & \\ & & & 1 \\ & 0 & & \boxed{U} \end{pmatrix} \begin{matrix} \\ \\ \\ \\ 11\ldots10 \\ 11\ldots11 \end{matrix}$$

$$= \sum_{x \in \{0,1\}^{n-1} \setminus \{1^{n-1}\}} |x\rangle\langle x| \otimes I + |1^{n-1}\rangle\langle 1^{n-1}| \otimes U$$

E.g. $C_1(X) = \text{CNOT}$. Since $HZH = X$,

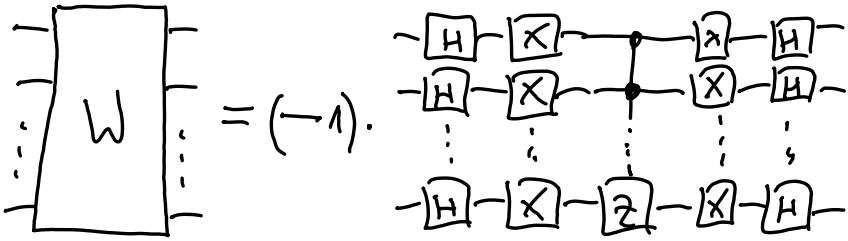
$$C_1(Z) = (I \otimes H)\,\text{CNOT}\,(I \otimes H).$$

$$\tilde{W} = -H^{\otimes n} X^{\otimes n} C_{n-1}(Z)\, X^{\otimes n} H^{\otimes n}$$

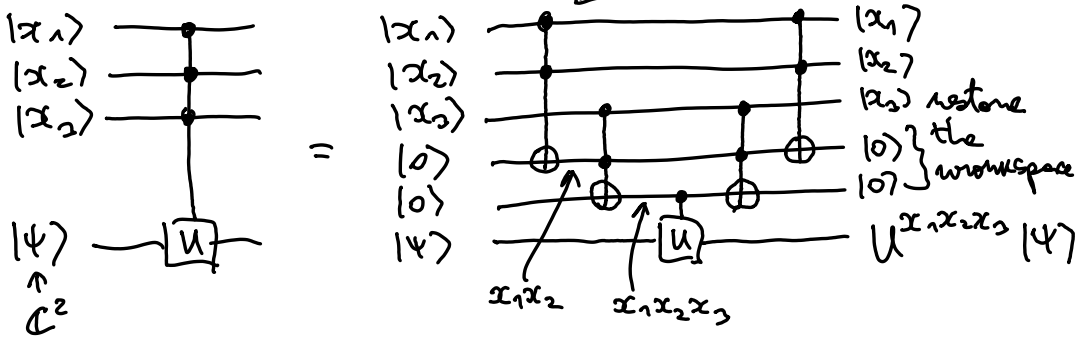
↑ can ignore this global phase

$C_{n-1}(U)|x_1\rangle|x_2\rangle\cdots|x_{n-1}\rangle|\psi\rangle$

$= \;$ ——— $\mathbb{1}$ ——— $U^{x_1\cdots x_{n-1}}|\psi\rangle$



How to implement $C_{n-1}(U)$? Toffoli $= C_2(X)$



This uses $O(n)$ Toffoli gates and $O(n)$ workspace.

Can we further decompose Toffoli into 1&2 qubit gates?

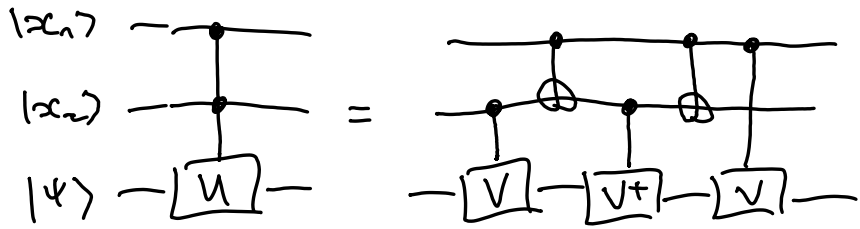
Exercise: Any 1-qubit operation can be decomposed as a sequence of Pauli ZYZ rotations.

Exercise: For any $U \in U(2)$, $C_1(U)$ can be implemented using only CNOT and Pauli rotations (ABC decomposition).

Special trick for $C_2(U)$:

For any $U \in U(2)$, $C_2(U)$ can be implemented as follows:

find $V \in U(2)$ s.t. $V^2 = U$

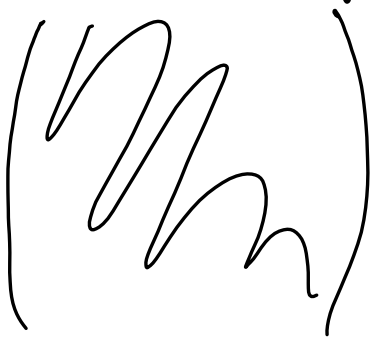


Only if $x_1 = x_2 = 1$ we apply $V^2 = U$.

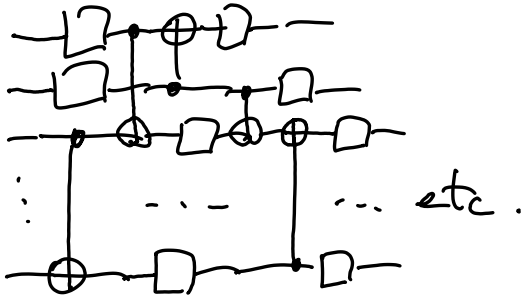
When $U = X$ we get a decomposition for Toffoli.

Quantum compiling

Given some n-qubit unitary $U \in U(2^n)$, how do we implement it as a sequence of only 1- and 2-qubit gts (only CNOTs, say)?



$\overset{?}{=}$



Main idea:

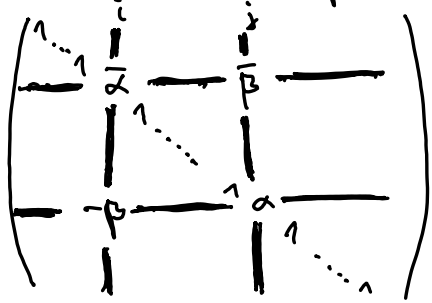
$$\underbrace{\frac{1}{\sqrt{|\alpha|^2+|\beta|^2}}\begin{pmatrix}\bar{\alpha} & \bar{\beta}\\ -\beta & \alpha\end{pmatrix}}_{\text{Unitary}}\begin{pmatrix}\alpha\\ \beta\end{pmatrix}_{\in\mathbb{C}^2} = \begin{pmatrix}\sqrt{|\alpha|^2+|\beta|^2}\\ 0\end{pmatrix}$$

Generalization of this is QR decomposition:

Let $G_{i,j}(\alpha,\beta) := \frac{1}{\sqrt{|\alpha|^2+|\beta|^2}}$ (Givens notation)

$\alpha,\beta \in \mathbb{C}$

$i,j \in \{0,1\}^n$



$$U = \begin{matrix}0\\1\\2\end{matrix}\begin{pmatrix}* & * & *\\ \alpha & * & *\\ \beta & * & *\end{pmatrix} \xrightarrow{G_{1,2}(\alpha,\beta)} \begin{pmatrix}\gamma & * & *\\ \delta & * & *\\ 0 & * & *\end{pmatrix} \xrightarrow{G_{0,1}(\gamma,\delta)} \begin{pmatrix}1 & 0 & 0\\ 0 & \varepsilon & *\\ 0 & \varphi & *\end{pmatrix}$$

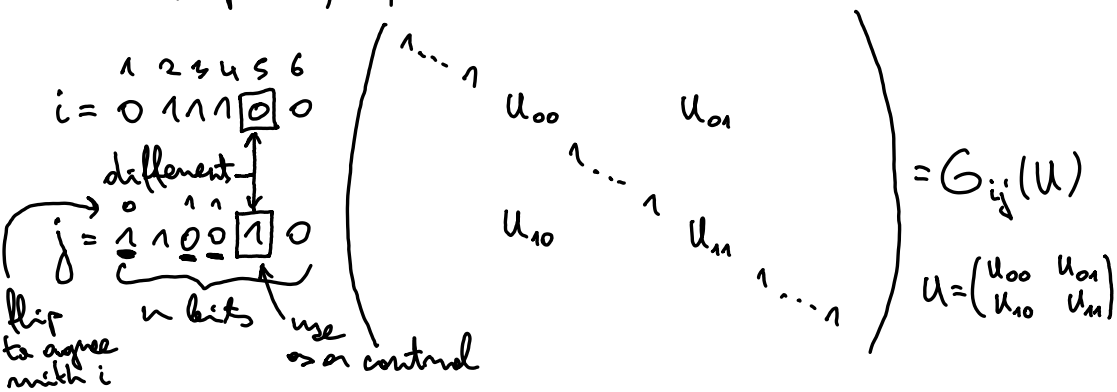
because unitary

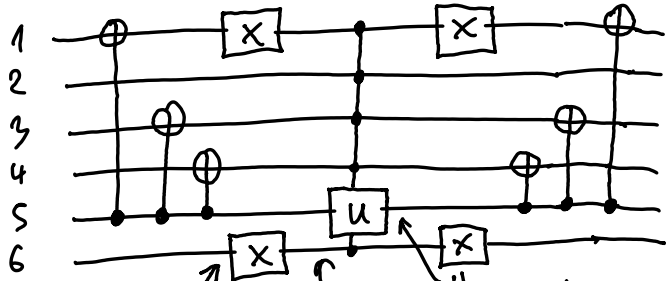
$$\xrightarrow{G_{1,2}(\varepsilon,\varphi)} \begin{pmatrix}1 & 0 & 0\\ 0 & 1 & 0\\ 0 & 0 & *\end{pmatrix} = e^{i\Theta} = \det(U)$$

Observation: we can decompose any n-qubit unitary as a sequence of 4^n Givens rotations.

How do we implement an arbitrary Givens rotation $G_{i,j}(\alpha,\beta)$?

1. Find a bit where $i,j \in \{0,1\}$ differ and use it as a control in the next step
2. Make all other differing bits of i and j the same by applying CNOTs that use them as target
3. Apply X to all bits where i and j are now 0
4. Apply U on the bit where i and j still differ, using all other bits as controls
5. Undo steps 3., 2., 1.





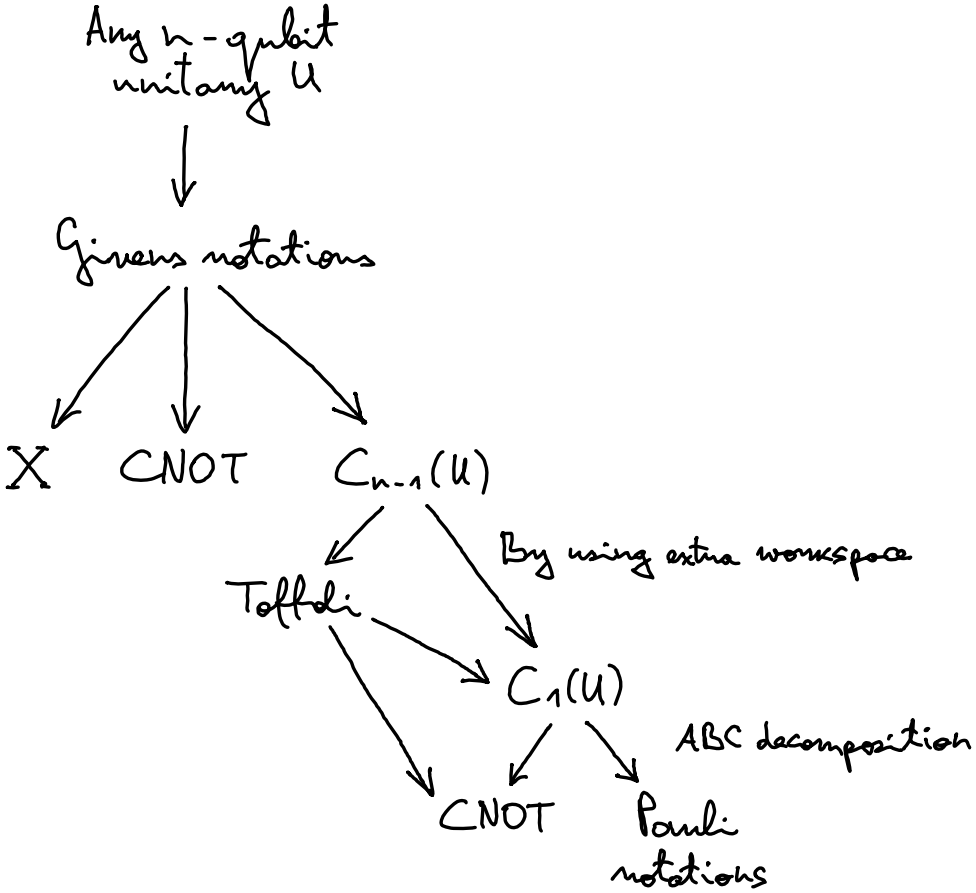
$= G_{i,j}(U)$

this is the qubit on which the two strings differ

$i = 0\,1\,1\,1\,0\,0$
$j \mapsto 0\,1\,1\,1\,1\,0$
agree

$i \mapsto 1\,1\,1\,1\,0\,1$
$j \mapsto 1\,1\,1\,1\,1\,1$

Summary: We can decompose any n-qubit unitary into Givens rotations, then decompose those into CNOT, X, $C_{n-1}(U)$, then using extra workspace decompose $C_{n-1}(U)$ into Toffoli and $C_1(U)$, then use the ABC decomposition.



Since a unitary matrix on n qubits is of size $2^n \times 2^n$, we need roughly 4^n parameters to describe it. Since each one- and two-qubit gate is of size 2×2 and 4×4, respectively, they contain only at most 4 and 16 parameters, respectively. It is thus inevitable that in general we end up with roughly $4^n/16$ gates, which is exponential in n.